DB

INTERVIEW PREPARATION

MySQL Basics

An Intermediate-Level Refresher with Practice Tasks & Solutions

CREATE TABLE & Constraints | SELECT / WHERE / ORDER BY | GROUP BY & HAVING | JOINs | Subqueries & CTEs | String & Date Functions | Indexes

`gold layer • fact_sales.csv • dim_customers.csv • dim_products.csv`

The Practice Dataset

Gold-layer star schema, loaded from CSV, used in every example below

All queries in this guide run against three gold-layer CSV files: one fact table and two dimension tables, forming a simple star schema.

Table	Role	Columns	Rows
`fact_sales.csv`	Fact — one row per sale	`sale_id, customer_id, product_id, quantity, sale_amount, sale_date`	200
`dim_customers.csv`	Dimension — customer attributes	`customer_id, customer_name, city, segment`	30
`dim_products.csv`	Dimension — product attributes	`product_id, product_name, category, unit_price`	20

Sample rows — fact_sales.csv

sale_id	customer_id	product_id	quantity	sale_amount	sale_date
1	14	108	3	612.30	2025-04-02
2	7	115	5	891.75	2025-06-19
3	22	103	2	140.00	2025-01-27

MYSQL

```mysql
-- 1. Create the database used for interview practice
CREATE DATABASE IF NOT EXISTS sales_gold;
USE sales_gold;

-- 2. Create the two dimension tables first (fact table references them)
CREATE TABLE dim_customers (
    customer_id   INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    city          VARCHAR(50),
    segment       VARCHAR(20)
);

CREATE TABLE dim_products (
    product_id    INT PRIMARY KEY,
    product_name  VARCHAR(100) NOT NULL,
    category      VARCHAR(50),
    unit_price    DECIMAL(10,2) NOT NULL
);

-- 3. Create the fact table with foreign keys pointing at both dimensions
CREATE TABLE fact_sales (
    sale_id       INT PRIMARY KEY,
    customer_id   INT,
    product_id    INT,
    quantity      INT NOT NULL,
    sale_amount   DECIMAL(10,2) NOT NULL,
```

```
    sale_date     DATE NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES dim_customers(customer_id),
    FOREIGN KEY (product_id)  REFERENCES dim_products(product_id)
);

-- 4. Load each CSV directly into its matching table
LOAD DATA INFILE '/var/lib/mysql-files/dim_customers.csv'
INTO TABLE dim_customers
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;   -- skip the CSV header row

LOAD DATA INFILE '/var/lib/mysql-files/dim_products.csv'
INTO TABLE dim_products
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

LOAD DATA INFILE '/var/lib/mysql-files/fact_sales.csv'
INTO TABLE fact_sales
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

SECURE_FILE_PRIV NOTE

MySQL only allows `LOAD DATA INFILE` from the directory set by the `secure_file_priv` system variable. Run `SHOW VARIABLES LIKE 'secure_file_priv';` to find that path, and copy your CSVs there first — or use MySQL Workbench's "Table Data Import Wizard" instead.

1 Tables, Data Types & Constraints

What interviewers expect you to know cold

Common MySQL data types: `INT`, `DECIMAL(p,s)` for exact money values, `VARCHAR(n)`, `DATE` / `DATETIME`, `BOOLEAN` (stored as TINYINT). Key constraints: `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE`, `DEFAULT`, `AUTO_INCREMENT`.

MYSQL

```
-- AUTO_INCREMENT generates surrogate keys automatically
CREATE TABLE employees (
    emp_id     INT AUTO_INCREMENT PRIMARY KEY,
    full_name  VARCHAR(100) NOT NULL,
    email      VARCHAR(100) UNIQUE,
    hire_date  DATE DEFAULT (CURRENT_DATE),
    salary     DECIMAL(10,2) CHECK (salary > 0)
);

-- ALTER TABLE modifies structure after creation
ALTER TABLE employees ADD COLUMN department VARCHAR(50);
ALTER TABLE employees MODIFY COLUMN full_name VARCHAR(150);
```

TASK 1

Write a `CREATE TABLE` statement for a table `returns` that logs product returns: `return_id` (auto-incrementing PK), `sale_id` (must reference `fact_sales.sale_id`), `return_reason` (text, required), and `return_date` (defaults to today).

SOLUTION

```
CREATE TABLE returns (
    return_id     INT AUTO_INCREMENT PRIMARY KEY,
    sale_id       INT NOT NULL,
    return_reason VARCHAR(255) NOT NULL,
    return_date   DATE DEFAULT (CURRENT_DATE),
    FOREIGN KEY (sale_id) REFERENCES fact_sales(sale_id)
);
```

2 SELECT, WHERE, ORDER BY, LIMIT

Filtering and sorting rows

MYSQL

```mysql
SELECT sale_id, customer_id, sale_amount, sale_date
FROM fact_sales
WHERE sale_amount BETWEEN 200 AND 800
  AND sale_date >= '2025-06-01'
ORDER BY sale_amount DESC
LIMIT 5;
```

OUTPUT

```
+---------+-------------+-------------+------------+
| sale_id | customer_id | sale_amount | sale_date  |
+---------+-------------+-------------+------------+
|     142 |          19 |      798.40 | 2025-07-02 |
|      88 |           3 |      760.00 | 2025-06-14 |
|     176 |          25 |      705.60 | 2025-08-09 |
+---------+-------------+-------------+------------+
```

TASK 2

Find the 3 most recent sales made in `'Cairo'` with a `quantity` of at least 4. Return `sale_id`, `city`, `quantity`, and `sale_date`, newest first.

SOLUTION

```mysql
SELECT f.sale_id, c.city, f.quantity, f.sale_date
FROM fact_sales f
JOIN dim_customers c ON f.customer_id = c.customer_id
WHERE c.city = 'Cairo'
  AND f.quantity >= 4
ORDER BY f.sale_date DESC
LIMIT 3;
```

Σ GROUP BY, HAVING & Aggregates

COUNT, SUM, AVG, MIN, MAX — and filtering groups

MYSQL

```
SELECT
    p.category,
    COUNT(*)                      AS num_sales,
    SUM(f.sale_amount)             AS total_revenue,
    ROUND(AVG(f.sale_amount), 2)     AS avg_sale
FROM fact_sales f
JOIN dim_products p ON f.product_id = p.product_id
GROUP BY p.category
HAVING COUNT(*) > 10
ORDER BY total_revenue DESC;
```

INTERVIEW REMINDER

`WHERE` filters rows before grouping; `HAVING` filters the aggregated groups afterward. Every non-aggregated column in `SELECT` must appear in `GROUP BY` (MySQL enforces this under `ONLY_FULL_GROUP_BY`, which is on by default since 5.7.5).

TASK 3

For each customer `segment`, return the total revenue and the number of distinct customers who bought something. Only include segments with more than 5 distinct customers.

SOLUTION

```
SELECT
    c.segment,
    SUM(f.sale_amount)              AS total_revenue,
    COUNT(DISTINCT f.customer_id) AS distinct_customers
FROM fact_sales f
JOIN dim_customers c ON f.customer_id = c.customer_id
GROUP BY c.segment
HAVING COUNT(DISTINCT f.customer_id) > 5
ORDER BY total_revenue DESC;
```

JOINs

INNER, LEFT, RIGHT, and self joins

MYSQL

```mysql
-- LEFT JOIN keeps every customer, even those with zero sales
SELECT c.customer_name, COUNT(f.sale_id) AS total_orders
FROM dim_customers c
LEFT JOIN fact_sales f ON c.customer_id = f.customer_id
GROUP BY c.customer_name
HAVING total_orders = 0;   -- customers who never purchased
```

INNER JOIN VS LEFT JOIN — CLASSIC INTERVIEW QUESTION

`INNER JOIN` returns only rows with a match on both sides. `LEFT JOIN` keeps every row from the left table, filling unmatched right-side columns with `NULL` — this is exactly how you find "customers with no orders" or "products never sold."

TASK 4

List every product that has **never** appeared in `fact_sales`. Return `product_name` and `category`.

SOLUTION

```mysql
SELECT p.product_name, p.category
FROM dim_products p
LEFT JOIN fact_sales f ON p.product_id = f.product_id
WHERE f.sale_id IS NULL;   -- no matching row means never sold
```

Subqueries & CTEs

MySQL 8.0+ supports WITH clauses (CTEs) and window functions

MYSQL (8.0+)

```sql
WITH customer_totals AS (
    SELECT customer_id, SUM(sale_amount) AS total_spend
    FROM fact_sales
    GROUP BY customer_id
)
SELECT c.customer_name, t.total_spend
FROM customer_totals t
JOIN dim_customers c ON t.customer_id = c.customer_id
WHERE t.total_spend > (
    SELECT AVG(total_spend) FROM customer_totals  -- scalar subquery
)
ORDER BY t.total_spend DESC;
```

TASK 5

Using a CTE, find the single top-spending customer in each `city`. Return `city`, `customer_name`, and `total_spend`.

SOLUTION

```sql
WITH customer_totals AS (
    SELECT c.city, c.customer_name, SUM(f.sale_amount) AS total_spend
    FROM fact_sales f
    JOIN dim_customers c ON f.customer_id = c.customer_id
    GROUP BY c.city, c.customer_name
),
ranked AS (
    SELECT *,
           RANK() OVER (
               PARTITION BY city ORDER BY total_spend DESC
           ) AS rnk
    FROM customer_totals
)
SELECT city, customer_name, total_spend
FROM ranked
WHERE rnk = 1;
```

Aa String & Date Functions

Everyday text and time manipulation in MySQL

MYSQL

```
SELECT
    UPPER(customer_name)                    AS name_upper,
    CONCAT(customer_name, ' - ', city)      AS name_and_city,
    SUBSTRING(customer_name, 1, 4)           AS short_name
FROM dim_customers
LIMIT 3;

SELECT
    sale_date,
    DATE_FORMAT(sale_date, '%Y-%m')  AS sale_month,
    DATEDIFF(CURDATE(), sale_date) AS days_ago,
    DATE_ADD(sale_date, INTERVAL 30 DAY) AS follow_up_date
FROM fact_sales
LIMIT 3;
```

TASK 6

Return total revenue grouped by **month** (format YYYY-MM), sorted chronologically.

SOLUTION

```
SELECT
    DATE_FORMAT(sale_date, '%Y-%m') AS sale_month,
    SUM(sale_amount) AS monthly_revenue
FROM fact_sales
GROUP BY sale_month
ORDER BY sale_month;
```

Indexes & Query Performance

What every intermediate MySQL interview asks about

An index is a separate, sorted data structure (typically a B-Tree) that lets MySQL locate rows without scanning the whole table. Primary keys and unique constraints create indexes automatically.

MYSQL

```mysql
-- Speed up frequent filtering/joining on customer_id
CREATE INDEX idx_fact_sales_customer ON fact_sales(customer_id);

-- A composite index helps queries that filter on BOTH columns together
CREATE INDEX idx_fact_sales_date_product ON fact_sales(sale_date, product_id);

-- EXPLAIN shows whether MySQL actually used an index for this query
EXPLAIN SELECT * FROM fact_sales WHERE customer_id = 14;
```

TRADE-OFF TO MENTION IN AN INTERVIEW

Indexes speed up `SELECT`/`WHERE`/`JOIN` but slow down `INSERT`/`UPDATE`/`DELETE`, since every write must also update the index structure. Index columns you filter or join on often — not every column.

TASK 7

You notice a dashboard query that filters `fact_sales` by both `sale_date` range and `product_id` is running slowly. What index would you create, and why does column order matter?

SOLUTION

```mysql
CREATE INDEX idx_sales_date_product ON fact_sales(sale_date, product_id);
```

A composite B-Tree index is sorted by the **first** column, then the second within each value of the first. Put `sale_date` first since the query filters on a date *range* — MySQL can still use the index for the trailing `product_id` equality check within that range. Reversing the order would make the `product_id` prefix far less selective for range scans.